```
»     def ensure_an_admin_remains
»       if User.count.zero?
»         raise Error.new "Can't delete last user"
»       end
»     end
  end
```

The key concept is the use of an exception to indicate an error when the user is deleted. This exception serves two purposes. First, because it's raised inside a transaction, it causes an automatic rollback. By raising the exception if the `users` table is empty after the deletion, we undo the delete and restore that last user.

Second, the exception signals the error back to the controller, where we use a `rescue_from` block to handle it and report the error to the user in the notice. If you want only to abort the transaction but not otherwise signal an exception, raise an `ActiveRecord::Rollback` exception instead, because this is the only exception that won't be passed on by `ActiveRecord::Base.transaction`:

rails80/depot_t/app/controllers/users_controller.rb

```
  def destroy
    @user.destroy!

    respond_to do |format|
      format.html { redirect_to users_path, status: :see_other,
        notice: "User #{@user.name} deleted" }
      format.json { head :no_content }
    end
  end

» rescue_from "User::Error" do |exception|
»   redirect_to users_url, notice: exception.message
» end
```

This code still has a potential timing issue: it's still possible for two administrators each to delete the last two users if their timing is right. Fixing this would require more database wizardry than we have space for here.